Dear interviewers:

Thank you for taking the time to review my report on question 2, "**Particle Flow Filter and Differentiable Particle Filter Part 1: From classical filters to particle flows**". We begin with a literature review of the evolution from basic Kalman filters to particle filters. Next, a point-to-point response is provided for each question. A codebase for each question is attached in the subfolder on GitHub. You can refer to more details in my github repo.

Yours sincerely,
Lixin Tu

Particle Flow Filter and Differentiable Particle Filter

Comment

This assignment will guide you from basic State Space Models (SSMs) filtering to advanced particle flow methods, culminating in an open-ended exploration of Differentiable Particle Filters (DPF) and HMC-based parameter inference. Your tasks are designed to both solidify your understanding of sequential inference and numerical stability of particle flows, and to enable end-to-end differentiable PF pipelines.

In your final report, please do make sure that you include a literature review of the methods that go beyond the methods discussed here to reflect your understanding of the problem.

Response:

This first part of the literature review covers the evolution from classical Kalman filtering to particle filtering, with a particular emphasis on particle–flow methods.

0.1 Linear and nonlinear Kalman filters

The Kalman filter is designed to compute an optimal estimate of the state of a noisy linear discrete–time system under certain assumptions by minimizing the mean-squared estimation error [Evangelidis and Parker, 2019]. Although the theoretical foundations of the Kalman filter are well established, implementing it practically remains challenging. Issues such as numerical instability and round-off errors frequently arise and can significantly degrade filter performance [Singh et al., 2023].

As many real–world systems are inherently nonlinear, extensions of the classical Kalman filter were developed to accommodate nonlinear dynamics and measurement models. The extended Kalman filter (EKF) linearizes the nonlinear system around the current estimate, enabling recursive state estimation in such conditions. The iterated extended Kalman filter (IEKF) further refines this idea by repeatedly performing the measurement update to improve estimation accuracy when the measurement function exhibits strong nonlinearity. Historically, the EKF played a critical role in the Apollo program for spacecraft localization, and it remains a widely used tool in robotics and navigation [Brossard et al., 2020]. More recent work integrates neural networks with EKF variants to adapt noise parameters or enhance pseudo–measurements. Examples include Brossard et al. [Brossard et al., 2020], who combined an Invariant EKF with a neural network, Roux et al. [Roux et al., 2021], who used a Right–Invariant EKF with a CNN for noise adaptation in projectile motion, and Zhou et al. [Zhou et al., 2022], who incorporated an attention-based recurrent network into an IEKF for automotive localization. While effective, such hybrid approaches typically require extensive training data, may not transfer reliably to specialized applications such as water-pipeline inspection, and can still suffer from long-term drift.

0.2 Particle filters

To overcome the limitations of linearization and Gaussian assumptions, particle filters were developed as a general framework for optimal Bayesian estimation in nonlinear and non-Gaussian systems. Particle filters approximate probability distributions using weighted samples and avoid the need for local linearization or crude functional approximations [Johansen, 2009]. Their main drawback lies in their computational cost, which is significantly higher than that of Kalman-filter–based methods. With the increase in available computing power, particle filters have nevertheless been adopted in real-time applications such as computer vision, financial econometrics, and target tracking. A central difficulty in particle filtering is the degeneracy problem, where repeated application of Bayes' rule—implemented as pointwise multiplication of likelihood and prior—causes most particles to obtain negligible weights. This reflects the curse of dimensionality and leads to a loss of sample diversity.

Particle-flow methods were introduced to address particle degeneracy by avoiding importance sampling altogether. The Exact Daum–Huang (EDH) flow [Daum et al., 2010] proposes a deterministic transformation that transports particles from the prior distribution to the posterior while circumventing the need for resampling, proposal densities from EKF, or explicit

evaluation of the conditional probability. Instead of applying Bayes' rule as a pointwise product, the EDH method operates on the log of the unnormalized posterior to improve numerical stability and eliminates reliance on Markov chain Monte Carlo techniques. Daum and Huang later emphasized [Daum and Huang, 2011] that the key to overcoming particle degeneracy is to compute the flow of the probability density itself and then propagate particles according to this induced flow.

Several variations of the Daum–Huang filter [Ding and Coates, 2012] have subsequently been proposed for discrete-time nonlinear filtering. These approaches propagate particle sets without importance sampling, using a log-homotopy that connects the prior to the posterior. The Local Exact Daum–Huang (LEDH) flow modifies the original EDH method by computing, for each particle individually, a linearization of the measurement function at the particle location, replacing the global linearization and removing the parallel EKF dependence. Furthermore, the LEDH method substitutes the EKF/UKF mean estimate with the state estimate produced directly by the EDH filter, resulting in a more localized and flexible flow.

Building on the particle-flow framework, Li et al. [Li and Coates, 2017] proposed a deterministic particle-flow approach to construct proposal distributions that closely approximate the target posterior. Their particle-flow particle filter (PFPF) incorporates both EDH-based and LEDH-based flows and defines invertible mappings that facilitate efficient weight computation. This design improves the effectiveness of particle-flow methods within a particle-filtering structure while reducing the computational burden relative to standard particle filters.

1 Part 1: From classical filters to particle flows

Comment 1.1

1) Warm-up. For those who are not familiar with filtering, do spend some time learning these basics before the next part.

I) Linear-Gaussian SSM with Kalman Filter

a) Implement the Kalman filter for a multidimensional linear-Gaussian SSM.
(Do not use tfp.distributions.LinearGaussianStateSpaceModel). Use synthetic data from a standard LGSSM, see examples 2 in [Doucet(09)].

Response:

Generally, a state-space model (SSM) with the hidden states being continuous can be described as:

$$\begin{aligned} x_k &= g(x_{k-1}, u_{k-1}, \omega_{k-1}) \\ z_k &= h(x_k, u_k, \upsilon_k) \end{aligned} \tag{1}$$

where x_k is the hidden state that evolves as a function g of the input u_k, the state x_k, and process noise ω. The output (or what we observe) is z_k which is some function h of the hidden state x_k, the input u_k, and some measurement (or sensor) error υ_k.

Considering the example 2 in [Doucet(09)], if we have a model where g and h are both linear functions and both error terms are Gaussian, we have a *Linear–Gaussian SSM (LG-SSM)*. More explicitly:

$$\begin{aligned} x_k &= A_{k-1}x_{k-1} + B_{k-1}u_{k-1} + \omega_{k-1} \\ z_k &= C_k x_k + D_k u_k + \upsilon_k \end{aligned} \tag{2}$$

Before employing a basic linear kalman filter (KF), we assume the ω_{k-1} and υ_k are mutually uncorrelated white Gaussain random processess, with zero-mean and covariance matrices having known value:

$$\mathbb{E}\left[w_n w_k^T\right] = \begin{cases} \Sigma_{\tilde{w}}, & n = k; \\ 0, & n \neq k. \end{cases} \quad \mathbb{E}\left[\upsilon_n \upsilon_k^T\right] = \begin{cases} \Sigma_{\tilde{\upsilon}}, & n = k; \\ 0, & n \neq k, \end{cases} \tag{3}$$

and $\mathbb{E}\left[w_k x_0^T\right] = 0$ for all k. The goal of the KF is to estimate the the posterior mean and covariance of x_k given past observations z_k and inputs u_k. Below are the main steps to design a linear Kalman filter [1,2,3,4] .

KF step 1a: State prediction time update. First, we compute the predicated state $\hat{x}_k^-$ as

$$\hat{x}_k^- = A_{k-1}\hat{x}_{k-1}^+ + B_{k-1}u_{k-1} \tag{4}$$

[1] Superscript "-" indicates a predicted quantity based only on past measurements.
[2] Superscript "+" indicates an estimated quantity based on both past and present measurements.
[3] Symbol "ˆ" indicates a predicted or estimated quantity.
[4] Symbol "˜" indicates an error.

KF step 1b: Prediction-error covariance time update. The covariance of the prediction error $\Sigma_{\tilde{x}_k}^-$ is

$$\Sigma_{\tilde{x}_k}^- = A_{k-1}\Sigma_{\tilde{x},k-1}^+ A_{k-1}^T + \Sigma_{\tilde{w}}. \tag{5}$$

KF step 1c: The predicted system output $\hat{z}_k$is

$$\hat{z}_k = C_k\hat{x}_k^- + D_k u_k, \tag{6}$$

KF step 2a: Estimator (Kalman) gain matrix L_k can be derived as

$$L_k = \Sigma_{\tilde{x},k}^- C_k^T \left[C_k\Sigma_{\tilde{x},k}^- C_k^T + \Sigma_{\tilde{v}}\right]^{-1} \tag{7}$$

KF step 2b: State estimate measurement update.

$$\hat{x}_k^+ = \hat{x}_k^- + L_k\left(z_k - \hat{z}_k\right). \tag{8}$$

KF step 2c: Estimation-error covariance measurement update. Finally, we update error covariance matrix $\Sigma_{\tilde{x},k}^+$ as

$$\begin{aligned}\Sigma_{\tilde{x},k}^+ &= \Sigma_{\tilde{x},k}^- - L_k C_k \Sigma_{\tilde{x},k}^- \\ &= \left(I - L_k C_k\right)\Sigma_{\tilde{x},k}^-\end{aligned} \tag{9}$$

The implementation codes in Python 3.11 can be found in the path "$./notebook/Q1\_1.ipynb$". The synthetic data from a standard LGSSM based on Example 2 in [Doucet(09)] is visualized in Figure 1.

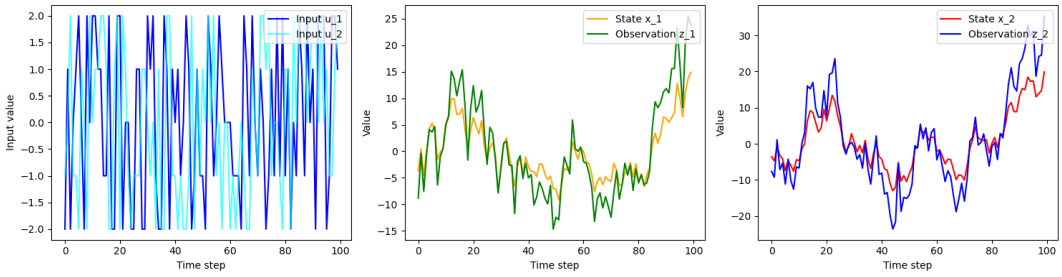


Figure 1: The synthetic data from a standard multidimensional linear-Gaussian SSM (left: inputs u_1 and u_2; middle: hidden states and observations for u_1; right: hidden states and observations for u_2.

With an implementation of the KF, a state error is calculated and is shown in Figure 2. Each estimated state error is plotted with its covariance bound.

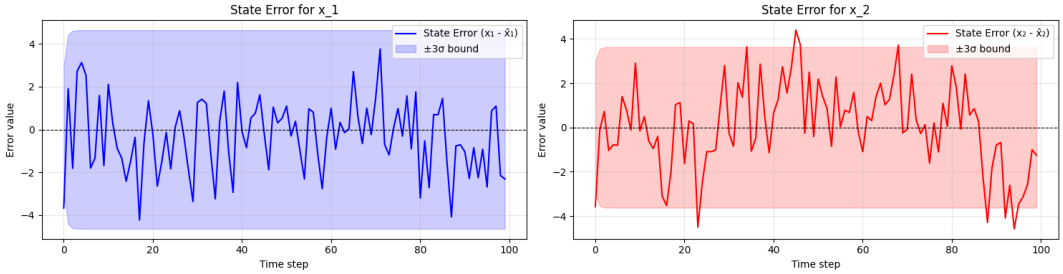


Figure 2: State estimation errors for the standard KF(left: KF results for u_1 with 99.0% errors within bounds; right: KF results for u_2 with 91.0% errors within bounds).

More analysis on the optimality and numerical stability will be discussed in the next response 1.2.

Comment 1.2

b) Analyze its filtering optimality and numerical stability: compare filtered means/covariances to the Kalman recursion; use Joseph stabilized covariance updates; discuss conditioning number.

Response:

The KF filter's optimality and numerical stability are analyzed according to (1) filtered means/covariances to the Kalman recursion; (2) employing Joseph stabilized covariance update; and (3) conditioning number discussion.

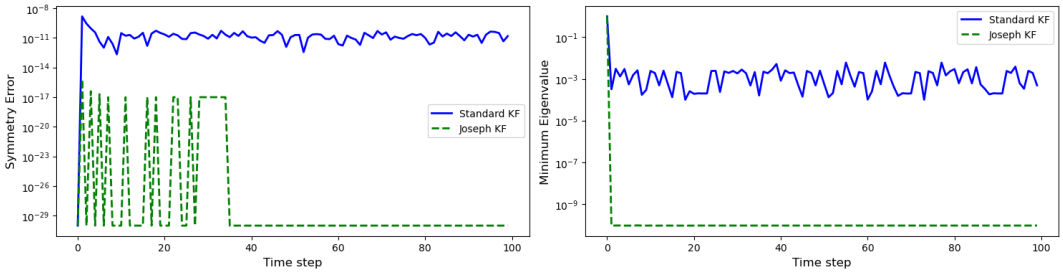


Figure 3: A comparison of symmetry error and eigenvalue.

1. **Compare filtered means/covariances to the Kalman recursion**

   In response 1.1, we assume that ω_{k-1} and υ_k are mutually uncorrelated white Gaussian random processes. A comparison between the true and estimated noises is shown in Table 1.

Table 1: Comparison of true and estimated noise mean and covariance

Noise Type	True Mean	Estimated Mean	True Covariance	Estimated Covariance
Process Noise ω_1	-0.091	0.042	2.231	4.342
Process Noise ω_2	0.267	0.373	1.350	6.298
Measurement Noise υ_1	0.141	-0.004	2.298	3.716
Measurement Noise υ_2	-0.011	0.291	2.463	4.660

2. **Employ Joseph stabilized covariance update**

   Within KF, covariance matrices $\Sigma^{+}_{\tilde{x},k}$ and $\Sigma^{-}_{\tilde{x},k}$ must remain (1) symmetric, and (2) positive definite (all eigenvalues strictly positive) at every time step. However, both conditions may be violated due to numerical floating-point rounding errors in a computer implementation. This will make the filter's confidence bounds incorrect and cause the KF to become unstable. One alternative is to use the Joseph-form covariance update, which is defined as below to replace the original estimation-error measurement-update equation in the **step 2c** (in response 1.1)

$$\Sigma^{+}_{\tilde{x},k} = [I - L_k C_k]\, \Sigma^{-}_{\tilde{x},k}\, [I - L_k C_k]^T + L_k \Sigma_{\tilde{v}} L_k^T \tag{10}$$

   Because the subtraction occurs in the squared term, the Joseph form guarantees a positive-definite result. Figure 3 compares the true state error between KF (in response 1.1) and KF with Joseph stabilized covariance update. Herein, an ill-conditioned Adjustment is made on both $\Sigma_{\widetilde{w}}$ and $\Sigma_{\tilde{v}}$ for a better visualization.

$$\Sigma_{\widetilde{w}} = \text{diag}([10^{-12}, 10^{12}]) = \begin{pmatrix} 10^{-12} & 0 \\ 0 & 10^{12} \end{pmatrix}, \quad \Sigma_{\tilde{v}} = \text{diag}([10^{-15}, 10^{5}]) = \begin{pmatrix} 10^{-15} & 0 \\ 0 & 10^{5} \end{pmatrix}$$

   The results demonstrate the superior numerical stability of the Joseph Stabilized KF. Specifically, the Joseph formulation eliminates the symmetry error seen in the Standard KF ($\sim 10^{-10}$) and maintains the positive definiteness of the covariance matrix by preventing the minimum eigenvalue from dropping below zero.

3. **Discuss conditioning number**.

   Besides, in order to quantify the goodness of the posterior state covariance matrix ($\Sigma^{+}_{\tilde{x},k}$), the condition number is usually used to provide an indication of the sensitivity of the solution . The condition number of $\Sigma^{+}_{\tilde{x},k}$ is given as

$$\kappa\left(\Sigma^{+}_{\tilde{x},k}\right) = \sigma_{\max}/\sigma_{\min} \tag{11}$$

   where $\sigma_{\max}$ and $\sigma_{\min}$ are the maximum and minimum singular values respectively [Evangelidis and Parker, 2019]. These can be obtained by performing the singular value decomposition (SVD) of $\Sigma^{+}_{\tilde{x},k}$. Figure 4 compares the condition number of standard KF and Joseph KF across time steps. It is noted that Joseph KF demonstrates a high condition number because the model is ill-conditioned. The Joseph KF maintains this very high condition number (around 10^9), which indicates numerical stability. By contrast, the standard KF algorithm holds a low condition number but fluctuates over the time steps.

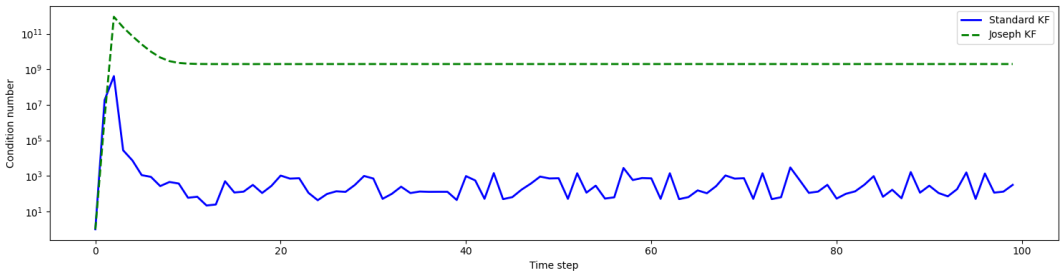


Figure 4: A comparison of the condition number.

Comment 1.3

II) Nonlinear/Non-Gaussian SSM with EKF/UKF and Particle Filter

a) Design a nonlinear and non-Gaussian SSM, you can use a stochastic volatility model (example 4 in [Doucet(09)]) or nonlinear tracking models(e.g. Range-Bearing observation model)

Response:

Consider LG-SSM defined in equation 2, and to make it a nonlinear and non-Gaussian SSM, we add a nonlinear term to the state x_k and add a non-Gaussian term to both process and measurement noises ω_k and υ_k. The new SSM becomes:

$$\begin{aligned} x_k &= A_{k-1}x_{k-1} + k_1 sin(x_{k-1}) + B_{k-1}u_{k-1} + \omega_{k-1} \\ z_k &= C_k x_k + k_2 sin(x_{k-1)} + D_k u_k + \upsilon_k \end{aligned} \tag{12}$$

where k_1 and k_2 are nonlinear coefficients. Herein, we assume that ω_{k-1} and υ_k are mutually uncorrelated, non-Gaussian heavy-tailed random processes following a Student's t-distribution with three degrees of freedom, with zero-mean and covariance matrices having known values.

The implementation codes in Python 3.11 can be found in the path "$./notebook/Q1\_3.ipynb$". The synthetic data is visualized in Figure 5. The distributions and Q-Q plots for both process and measurement noises are shown in Figure 6 to illustrate the non-Gaussian characteristic of the SSM model ($k_1 = 0.5, k_2 = 0.3$).

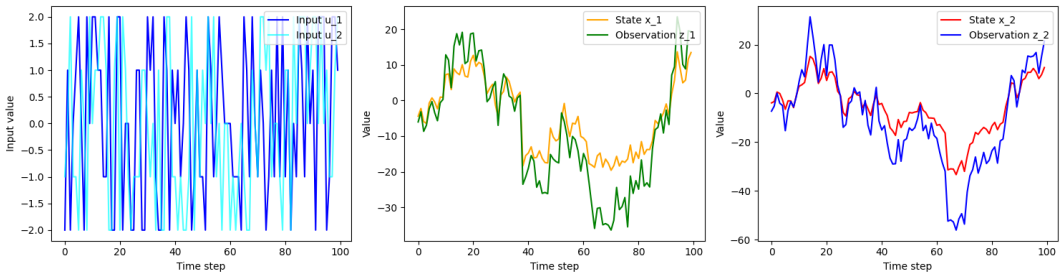


Figure 5: The synthetic data from a non-linear and non-Gaussian SSM (left: inputs u_1 and u_2; middle: hidden states and observations for u_1; right: hidden states and observations for u_2.

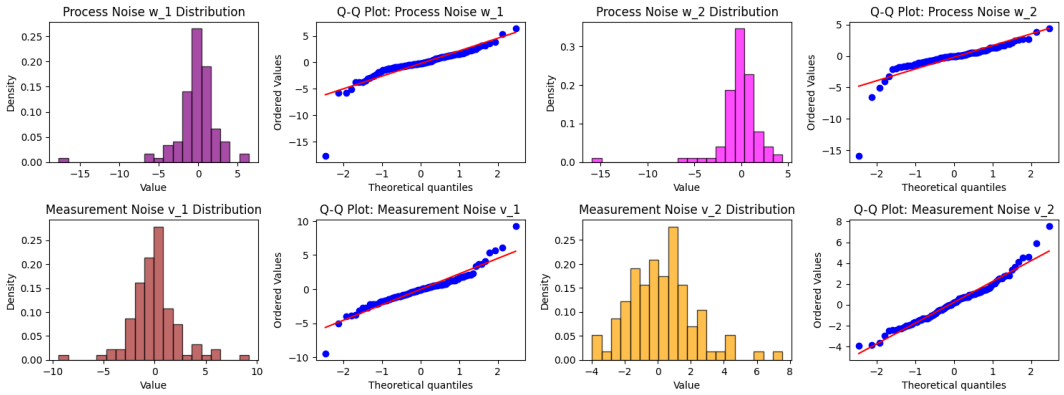


Figure 6: The non-Gaussian noise characteristic from the SSM model defined in the equation 28.

Comment 1.4

b) Implement the Extend Kalman filter (EKF) and Unsent Kalman Filter(UKF), discuss linearization accuracy limits and sigma point failures under strong nonlinearity.

Response:

(1) Implmentation of EKF and UKF

To solve nonlinear systems defined in the equation (28), there are three basic KF generalizations.

- Extended Kalman filter (EKF): it analyses the linearization of the model at each point in time.
- Sigma-point (unscented) Kalman filter (UKF): it analyses statistical/empirical linearisation of the model at each point in time. It often performs better than EKF at the same computational complexity.
- Particle filters: More precise, but often thousands of times more computations required than either EKF or UKF.

The standard KF results for the SSM defined in the equation 28 are shown in Figure 7 for a baseline reference.

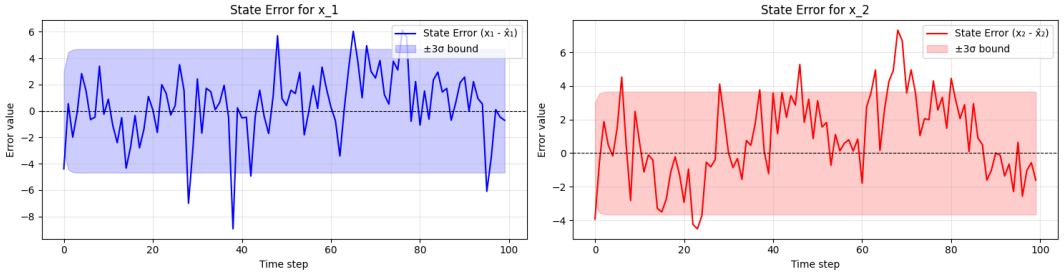


Figure 7: State estimation errors for the standard KF(left: KF results for u_1 with 90.0% errors within bounds; right: KF results for u_2 with 84.0% errors within bounds).

EKF makes two simplifying assumptions when applying: (1) When computing estimates of the output of a nonlinear function, EKF assumes $\mathbb{E}[f(x)] \approx f(\mathbb{E}[x])$. (2) When computing covariance estimates, EKF uses Taylor-series expansion to linearize the system equations around the present operating point. Recall the standard KF derivation in equations (2) to (9),

EKF step 1a switches to use linear approximation $\hat{A}_{k-1}$(e.g, Taylor series) to estimate the state, which gives

$$\hat{x}_k^- = \hat{A}_{k-1}\hat{x}_{k-1}^+ + B_{k-1}u_{k-1} \tag{13}$$

EKF step 1b: The covariance of the prediction error $\Sigma_{\tilde{x}_k}^-$ is

$$\Sigma_{\tilde{x}_k}^- = \hat{A}_{k-1}\Sigma_{\tilde{x},k-1}^+\hat{A}_{k-1}^T + \Sigma_{\tilde{w}}. \tag{14}$$

EKF step 1c also switches to use linear approximation $\hat{C}_{k-1}$(e.g, Taylor series) to estimate the system output, which gives

$$\hat{z}_k = \hat{C}_k \hat{x}_k^- + D_k u_k, \tag{15}$$

EKF step 2a: Estimator (Kalman) gain matrix L_k is derived as

$$L_k = \Sigma_{\tilde{x},k}^- \hat{C}_k^T \left[\hat{C}_k \Sigma_{\tilde{x},k}^- \hat{C}_k^T + \Sigma_{\tilde{v}}\right]^{-1} \tag{16}$$

EKF step 2b: State estimate measurement update.

$$\hat{x}_k^+ = \hat{x}_k^- + L_k \left(z_k - \hat{z}_k\right). \tag{17}$$

EKF step 2c: Finally, the updated error covariance matrix is $\Sigma_{\tilde{x},k}^+$ as

$$\Sigma_{\tilde{x},k}^+ = \Sigma_{\tilde{x},k}^- - L_k(\hat{C}_k \Sigma_{\tilde{x},k}^- \hat{C}_k^T + \Sigma_{\tilde{v}}) L_k^T \tag{18}$$

The EKF results for the SSM defined in the equation 28 are shown in Figure 8.

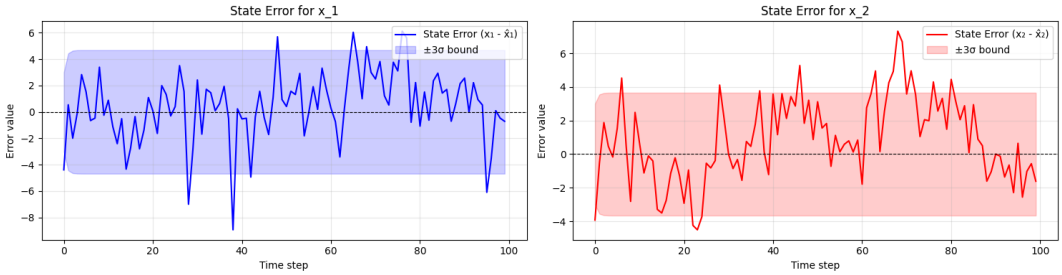


Figure 8: State estimation errors for the EKF(left: EKF results for u_1 with 91.0% errors within bounds; right: EKF results for u_2 with 90.0% errors within bounds).

On the other hand, **UKF** is a "sigma-point" (SP) approach that can improve mean and covariance propagation through nonlinear equations. It can propagate the uncertainty of the input random variable (RV) to the output of the model's (possibly) nonlinear state and output equations (add reference here).

UKF step 1a form augmented prior state prediction and covariance:

$$\hat{x}_{k-1}^{a,+} = \left[\left(\hat{x}_{k-1}^+\right)^T, \bar{w}^T, \bar{v}^T\right]^T, \Sigma_{\tilde{x},k-1}^{a,+} = \text{diag}\left(\Sigma_{\tilde{x},k-1}^+, \Sigma_{\tilde{w}}, \Sigma_{\tilde{v}}\right) \tag{19}$$

These factors are used to generate the $p+1$ augmented sigma points:

$$\mathcal{X}_{k-1}^{a,+} = \left\{\hat{x}_{k-1}^{a,+}, \hat{x}_{k-1}^{a,+} + \gamma\sqrt{\Sigma_{\tilde{x},k-1}^{a,+}}, \hat{x}_{k-1}^{a,+} - \gamma\sqrt{\Sigma_{\tilde{x},k-1}^{a,+}}\right\}. \tag{20}$$

Then, the present state prediction can be computed as

$$\hat{x}_k^- \approx \sum_{i=0}^{p} \alpha_i^{(\mathrm{m})} g\left(\mathcal{X}_{k-1,i}^{x,+}, u_{k-1}, \mathcal{X}_{k-1,i}^{w,+}\right) = \sum_{i=0}^{p} \alpha_i^{(\mathrm{m})} \mathcal{X}_{k,i}^{x,-} \tag{21}$$

UKF step 1b: The covariance of the prediction error $\Sigma_{\tilde{x}_k}^-$ is

$$\Sigma_{\tilde{x},k}^- = \sum_{i=0}^{p} \alpha_i^{(\mathrm{c})} \left(\mathcal{X}_{k,i}^{x,-} - \hat{x}_k^-\right)\left(\mathcal{X}_{k,i}^{x,-} - \hat{x}_k^-\right)^T \tag{22}$$

UKF step 1c computes the predicted output $\hat{z}_k$ using sigma points descibring state and sensor noise.

$$\hat{z}_k \approx \sum_{i=0}^{p} \alpha_i^{(\mathrm{m})} h\left(\mathcal{X}_{k,i}^{x,-}, u_k, \mathcal{X}_{k-1,i}^{v,+}\right) = \sum_{i=0}^{p} \alpha_i^{(\mathrm{m})} \mathcal{Z}_{k,i} \tag{23}$$

UKF step 2a: Estimator (Kalman) gain matrix L_k is derived from covariance matrices

$$\begin{aligned} \Sigma_{\tilde{z},k} &= \sum_{i=0}^{p} \alpha_i^{(\mathrm{c})} \left(\mathcal{Z}_{k,i} - \hat{z}_k\right)\left(\mathcal{Z}_{k,i} - \hat{z}_k\right)^T \\ \Sigma_{\tilde{x}\tilde{z},k}^- &= \sum_{i=0}^{p} \alpha_i^{(\mathrm{c})} \left(\mathcal{X}_{k,i}^{x,-} - \hat{x}_k^-\right)\left(\mathcal{Z}_{k,i} - \hat{z}_k\right)^T \end{aligned} \tag{24}$$

$$L_k = \Sigma^{-}_{\tilde{x}\tilde{z},k}\Sigma^{-1}_{\tilde{z},k} \tag{25}$$

UKF step 2b: State estimate measurement update.

$$\hat{x}^{+}_k = \hat{x}^{-}_k + L_k\left(z_k - \hat{z}_k\right). \tag{26}$$

UKF step 2c: Finally, the updated error covariance matrix is $\Sigma^{+}_{\tilde{x},k}$ as

$$\Sigma^{+}_{\tilde{x},k} = \Sigma^{-}_{\tilde{x},k} - L_k\Sigma_{\tilde{z},k}L^T_k \tag{27}$$

The UKF results for the SSM defined in the equation 28 are shown in Figure 9.

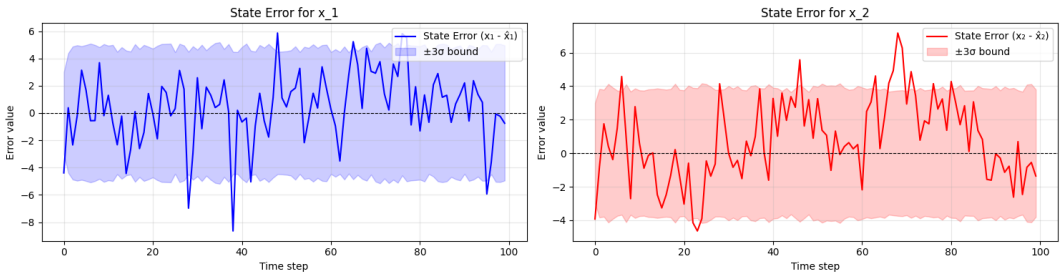
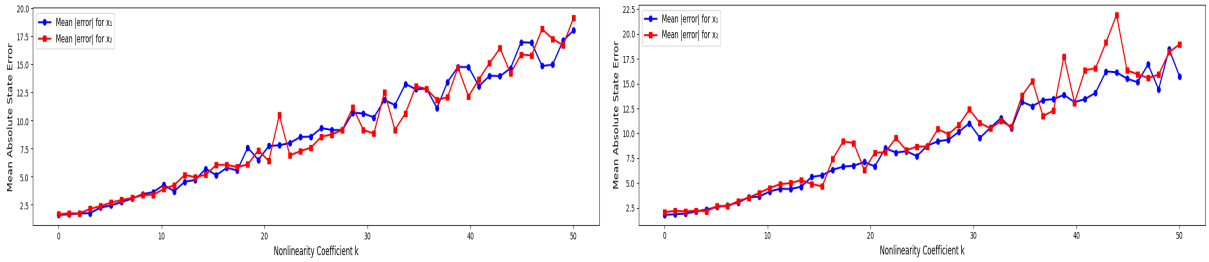


Figure 9: State estimation errors for the UKF(left: UKF results for u_1 with 91.0% errors within bounds; right: UKF results for u_2 with 85.0% errors within bounds).

(2) Linearization accuracy limits and sigma point failures under strong nonlinearity

We also adjust the model's nonlinearity according to the set nonlinearity coefficients k, ranging from 0.0 (linear) to 50.0 (strong nonlinearity). The mean absolute error for both state dimensions is computed for evaluation. As shown in Figure 10a, as the true system becomes more nonlinear, the linear approximation used by the EKF becomes less accurate, leading to larger estimation errors. The performance of UKF also deteriorates as the nonlinearity becomes stronger.



(a) EKF ($k_1 = k_2 = k$) (b) UKF($k_1 = k_2 = k$)

Figure 10: Mean absolute state error at different system nonlinearity

Figure 11 further demonstrates an analysis of ULF sigma point failure under different system nonlinearity. It could be found that when $k = 0$ (linear), the UKF estimate is exact. When $k = 3.0$ (mild nonlinearity), the UKF estimate (green) begins to diverge from the true distribution. Finally, when $k = 10.0$ (strong nonlinearity), the transformed sigma points (red dots) are severely distorted, causing the UKF's computed mean and covariance (green ellipse) to be completely erroneous compared to the ground truth. This demonstrates that the strong nonlinearity violates the UKF's core assumption that the transformed sigma points can still be reasonably approximated by a Gaussian distribution, which leads to estimation failure.

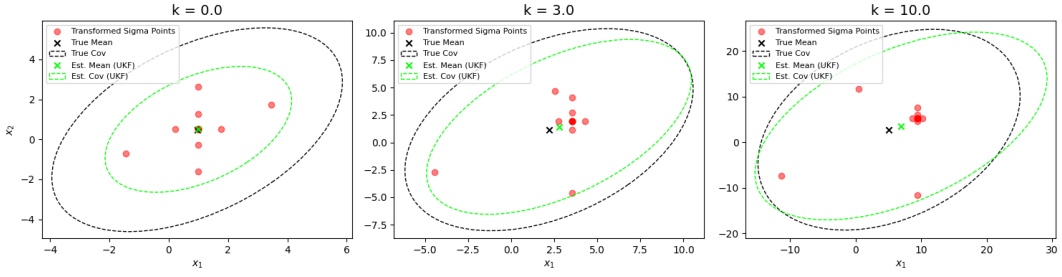


Figure 11: A visualization of sigma points failure analysis for UKF (set nonlinear coefficient $k_1 = k_2 = k$ for simplicity

Comment 1.5

c) Implement a standard particle filter for your model. (Do not use the tfp.experimental.mcmc.particle_filter). Visualize and discuss issues such as particle degeneracy.

Response:

Reconsider the nonlinear and non-Gaussian model defined in equation (28),

$$\begin{aligned} x_k &= A_{k-1}x_{k-1} + k_1 sin(x_{k-1}) + B_{k-1}u_{k-1} + \omega_{k-1} \\ z_k &= C_k x_k + k_2 sin(x_{k-1)} + D_k u_k + \upsilon_k \end{aligned} \tag{28}$$

where k_1 and k_2 are nonlinear coefficients, a standard particle filter seeks to evaluate

$$\begin{aligned} f\left(x_k \mid \mathbb{Z}_{k-1}\right) &= \int_{R_{x_{k-1}}} f\left(x_k \mid x_{k-1}\right) f\left(x_{k-1} \mid \mathbb{Z}_{k-1}\right) \mathrm{d}x_{k-1} \\ f\left(x_k \mid \mathbb{Z}_k\right) &= \frac{f\left(z_k \mid x_k\right) f\left(x_k \mid \mathbb{Z}_{k-1}\right)}{f\left(z_k \mid \mathbb{Z}_{k-1}\right)} \end{aligned} \tag{29}$$

The objective of the particle filter is to estimate either the joint or marginal pdf: $f\left(\mathbb{X}_k \mid \mathbb{Z}_k\right)$ or $f\left(x_k \mid \mathbb{Z}_k\right)$. The joint pdf can be expressed as

$$\hat{f}\left(\mathbb{X}_k^{(i)} \mid \mathbb{Z}_k\right) = \sum_{i=1}^{N_p} w_k^{(i)} \delta\left(\mathbb{X}_k - \mathbb{X}_k^{(i)}\right) \tag{30}$$

where the weight is updated according to

$$\widetilde{w}_k^{(i)} = \frac{f\left(z_k \mid x_k^{(i)}\right) f\left(x_k^{(i)} \mid x_{k-1}^{(i)}\right)}{q\left(x_k^{(i)} \mid x_{k-1}^{(i)}, z_k\right)} \widetilde{w}_{k-1}^{(i)} \tag{31}$$

The marginal pdf can be obtained by integrating the joint pdf $\hat{f}\left(\mathbb{X}_k^{(i)} \mid \mathbb{Z}_k\right)$

$$\hat{f}\left(x_k \mid \mathbb{Z}_k\right) = \sum_{i=1}^{N_p} w_k^{(i)} \delta\left(x_k - x_k^{(i)}\right) \tag{32}$$

The most basic general particle-filter algorithm, known as "sequential importance sampling (SIS)", can be written as

$$\left\{x_k^{(i)}, w_k^{(i)}\right\}_{i=1}^{N_p} = \mathrm{SIS}\left(\left\{x_{k-1}^{(i)}, w_{k-1}^{(i)}\right\}_{i=1}^{N_p}, z_k\right) \tag{33}$$

basic PF step 1a: Draw new particle value $x_k^{(i)}$ randomly

$$x_k^{(i)} \sim q\left(x_k \mid x_{k-1}^{(i)}, z_k\right) \tag{34}$$

basic PF step 1b: Calculate unnormalized weight $\widetilde{w}_k^{(i)}$ for this particle

$$\widetilde{w}_k^{(i)} = \frac{f\left(z_k \mid x_k^{(i)}\right) f\left(x_k^{(i)} \mid x_{k-1}^{(i)}\right)}{q\left(x_k^{(i)} \mid x_{k-1}^{(i)}, z_k\right)} w_{k-1}^{(i)} \tag{35}$$

basic PF step 1b: repeat step 1a and step 1b fpr N_p times

basic PF step 2a: normalized the weights $w_k^{(i)}$ for all prticles

$$w_k^{(i)} = \frac{\widetilde{w}_k^{(i)}}{\sum_{i=1}^{N_p} \widetilde{w}_k^{(i)}}. \tag{36}$$

basic PF step 2b: State estimate measurement $\hat{x}_k^+$ update

$$\hat{x}_k^+ = \sum_{i=1}^{N_p} w_k^{(i)}\, x_k^{(i)}. \tag{37}$$

basic PF step 2c: Finally, the updated error covariance matrix $\Sigma^{+}_{\tilde{x},k}$

$$\Sigma^{+}_{\tilde{x},k} = \sum_{i=1}^{N_p} w_k^{(i)} \left(x_k^{(i)} - \hat{x}_k^{+}\right)\left(x_k^{(i)} - \hat{x}_k^{+}\right)^{\top}. \tag{38}$$

Fig. 12 shows particle trajectories output with true state trajectory overlaid, where particles represent many possible trajectories to choose from. It is found that when the time step is getting higher, low particle density near the true state. In addition, Fig. 13 demonstrates SIS-estimated marginal PDFs using kernel functions. It is found that the SIS estimate starts out acceptable, but quickly collapses to a single likely trajectory, which is not what we expected. This is a vulnerability of the basic SIS algorithm: particle trajectories can become irrelevant, a phenomenon also called “particle degeneracy".

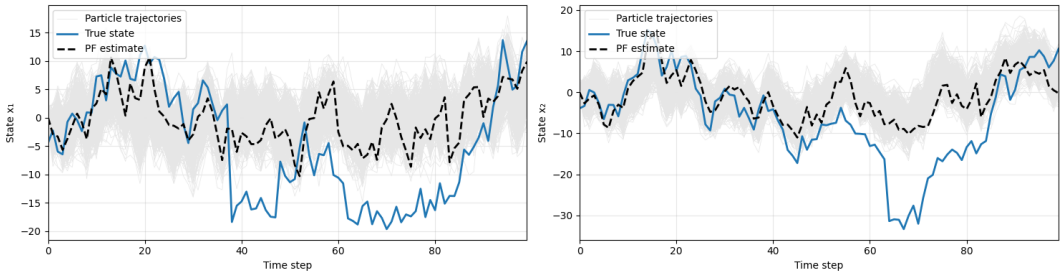


Figure 12: A comparison between particle trajectories with the true state.

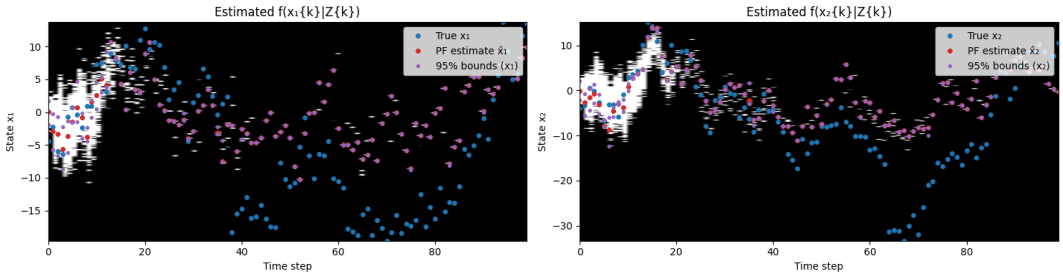


Figure 13: Colormaps of SIS estimated pdfs.

The particle weights plots in Fig. 14 further visulizes that all weighs start at the same value at the very beginning, but very quickly it is found that one particle "take over" and the other particles’ weights decay to values close to zero. Therefore, those particles have no influence on the results. In the standard deviation of weights plot, the increasing value shows taht weights ar3e becoming very unequal, and some particles are being ignored. THis is undersiable as we prefer for all particles to participate more-or-less equally in the compuation of the estimated pdf.

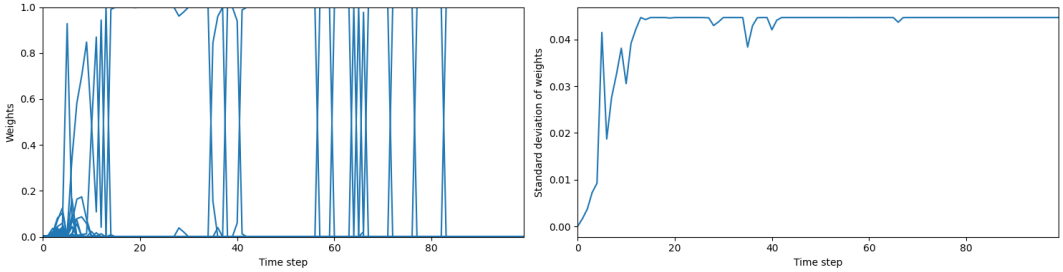


Figure 14: Examining the particle weights (left: particle weights on natual scales; right: standard deviation of the weights).

Comment 1.6

d) Compare PF and EKF/UKF performance. How to evaluate your SSMs? What metrics can you use? In practice, we care about the runtime and memory, could you also compare the runtime and peak memory(CPU/GPU RAM) for each SSM?

Response:

The PF results for the SSM defined in the equation 28 are shown in Figure 15.

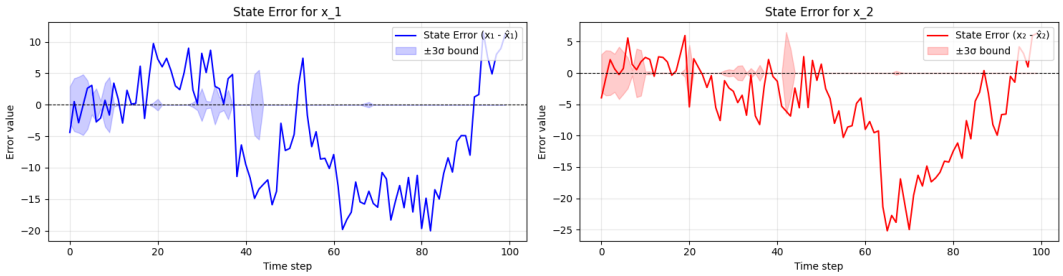


Figure 15: State estimation errors for the PF (left: PF results for u_1 with 12.0% errors within bounds; right: UKF results for u_2 with 7.0% errors within bounds).

Comparing to the performance for EKF in Fig. 8 and UKF in Fig. 9, RMSE value for each state is generally used for evaluation (add two references). In particular,

$$\mathrm{RMSE}(x_j) = \sqrt{\frac{1}{k}\sum_{t=1}^{k}\left(x_t^{(j)} - \hat{x}_t^{(j)}\right)^2}, \qquad j \in \{1, 2\} \tag{39}$$

where k is the time step. Table 2 summarizes the RMSE results for EKF/UKF and PF. Based on the current settings, the EKF performs the best in our experiments.

Table 2: Comparison of RMSE for different filters. **Bold** indicates the best.

Filter	**RMSE**(x_1)	**RMSE**(x_2)
Extended Kalman Filter (EKF)	**2.257**	**1.848**
Unscented Kalman Filter (UKF)	2.595	2.554
Basic Particle Filter (SIS)	9.007	9.093

Besides, from the employment perspective, the runtime and peak memory(CPU/GPU RAM) for each SSM is considered and compared in Table 3. It is noted that the Particle Filter shows significantly higher runtime and memory usage compared to EKF and UKF due to its sampling-based nature.

Table 3: Runtime and peak memory usage comparison of different filters. **Bold** indicates the best.

Filter	**Runtime (s)**	**Peak Memory (MB)**
Extended Kalman Filter (EKF)	**0.0244**	**0.04**
Unscented Kalman Filter (UKF)	0.1240	0.06
Basic Particle Filter (SIS)	12.6329	1.22

Comment 1.7

2) Deterministic and Kernel Flows

a) Study the Exact Daum-Huang (EDH) flow and Local Exact Daum-Huang (LEDH) flow, (see [Daum(10)] and [Daum(11)]), and the invertible particle flow particle filter (PF-PF) framework (see Li(17)). Replicates the main results in Li(17).

Response:

To replicate the main results reported in [Li and Coates, 2017], we adopted the same multi-target acoustic tracking simulation setup. The replicated results are presented in Fig. 16 and Table 4.

The left panel of Fig. 16 displays the average optimal mass transfer (OMAT) metric at each time step. All methods except LEDH achieve relatively small average tracking errors. The boxplots in the right panel, which summarize the average OMAT over time, show a similar performance trend. The middle panel and Table 4 further indicate that both PFPF algorithms incur

only a slight increase in execution time compared with EDH and LEDH, demonstrating that the weight-update procedure remains computationally efficient.

In Table 4, the EDH and PFPF_EDH methods exhibit comparable accuracy and runtime, while LEDH and PFPF_LEDH produce larger tracking errors and substantially higher computational costs.

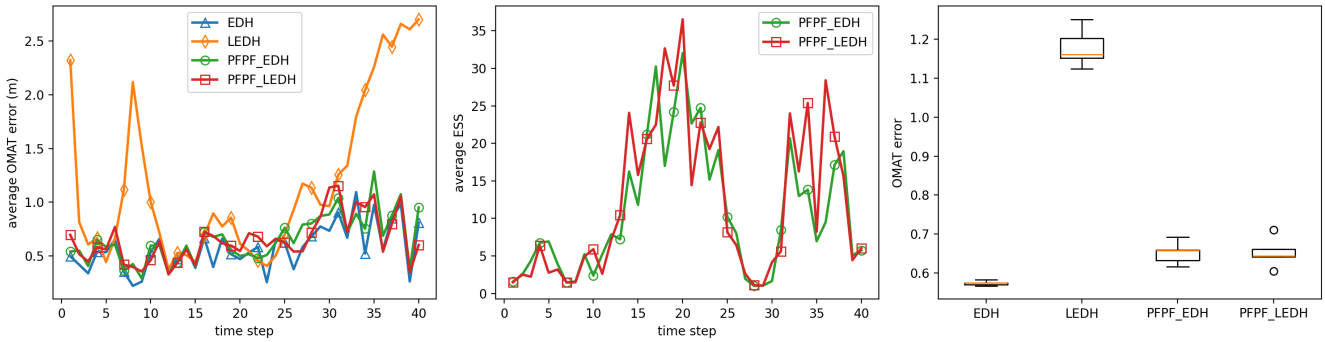


Figure 16: Main results replicated in the multi-target acoustic tracking example [Li and Coates, 2017] (left: Average OMAT errors at each time; middle: Average effective sample size at each time; right: boxplots of average OMAT error)

Table 4: Average OMAT metrics and execution time over time steps.

Algorithm	**Particle num.**	**Avg. OMAT (m)**	**Exec. time (s)**
EDH	500	0.5733	2.0989
LEDH	500	1.1784	701.3275
PFPF_EDH	500	0.6512	2.3272
PFPF_LEDH	500	0.6518	832.2722

Comment 1.8

b) Implement the kernel-embedded particle flow filter in an RKHS following Hu(21). Then compare the scalar kernel and diagonal matrix-valued kernel. Use experiments to demonstrate the matrix-valued kernel can prevent collapse of observed-variable marginals in high-dimensional, plot similar figures as figure 2-3 in Hu(21).

Response:

The particle flow particle filter (PFPF) avoids the long-standing weight-degeneracy problem in particle filters and has great potential for application in high-dimensional systems. The PFPF sequentially pushes particles from the prior to the posterior distribution, without changing each particle's weight. The PFPF assumes the particle flow is embedded in a Reproducing Kernel Hilbert Space (RKHS), so that a practical solution for the particle flow is obtained.

Many studies have shown that a scalar kernel fails in high-dimensional, sparsely observed settings [Poterjoy, 2015]. To resolve this issue, Hu and Van Leeuwen [Hu and van Leeuwen, 2021] proposed a new matrix-valued kernel to prevent the collapse of marginals in a high-dimensional system. We conducted a similar experiment (a 1,000-dimensional Lorenz 96 model) with Hu (21) to demonstrate that the matrix-valued kernel can prevent the collapse of observed-variable marginals in high dimensions. Figure 17 shows the posterior marginal distribution of the variable x_{19} (unobserved component) and x_{20} (observed component) after the first data assimilation update. The result is similar to the case in Hu(21), where the matrix-valued kernel can keep particles away from each other, preventing the collapse in the observed component, while the particles collapse for x_{20} using the scalar kernel.

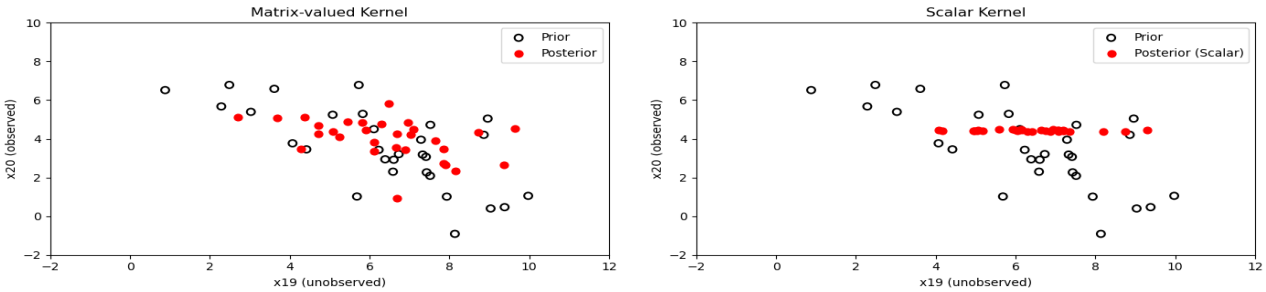


Figure 17: The prior and posterior marginal distribution of the variable x_{19} (unobserved component) and x_{20} (observed component) before and after the first data assimilation update (t=20), using the (a) matrix-valued kernel and (b) scalar kernel.

Comment 1.9

c) Compare EDH, LEDH, and kernel PFF on the SSM you designed in last particle filter question. Analyze when each method excels or fails(nonlinearity, observation sparsity, dimension, conditioning). Include stability diagnostics(flow magnitude, Jacobian conditioning).

Response:

Your Response 1.9
EDH is from comment 1.7, LEDH is from comment 1.7 as well

2 Response to reviewer 2

Comment 2.1

Comments 2.1

Response:

Your Response 2.1

Comment 2.2

Comments 2.2

Response:

Your Response 2.2

Comment 2.3

Comments 2.3

Response:

Your Response 2.3

Comment 2.4

Comments 2.4

Response:

Your Response 2.4

3 References

[Daum(10)] Daum, Fred, Jim Huang, and Arjang Noushin. "Exact particle flow for nonlinear filters." Signal processing, sensor fusion, and target recognition XIX. Vol. 7697. SPIE, 2010.

[Daum(11)] Daum, Fred, and Jim Huang. "Particle degeneracy: root cause and solution." Signal Processing, Sensor Fusion, and Target Recognition XX. Vol. 8050. SPIE, 2011.

[Li(17)] Li, Yunpeng, and Mark Coates. "Particle filtering with invertible particle flow." IEEE Transactions on Signal Processing 65.15 (2017): 4102-4116.

[Hu(21)] Hu, Chih-Chi, and Peter Jan Van Leeuwen. "A particle flow filter for high-dimensional system applications." Quarterly Journal of the Royal Meteorological Society 147.737 (2021): 2352-2374.

[Dai(21)] Dai, Liyi, and Fred Daum. "A new parameterized family of stochastic particle flow filters." arXiv preprint arXiv:2103.09676 (2021).

[Dai(22)] Dai, Liyi, and Fred Daum. "Stiffness mitigation in stochastic particle flow filters." IEEE Transactions on Aerospace and Electronic Systems 58.4 (2022): 3563-3577.

[Corenflos(21)] Corenflos, Adrien, et al. "Differentiable particle filtering via entropy-regularized optimal transport." International Conference on Machine Learning. PMLR, 2021.

[Chaudhari(25)] Chaudhari, Shreyas, Srinivasa Pranav, and José MF Moura. "GradNetOT: Learning Optimal Transport Maps with GradNets." arXiv preprint arXiv:2507.13191 (2025).

[Jha(25)] Jha, Prashant K. "From Theory to Application: A Practical Introduction to Neural Operators in Scientific Computing." arXiv preprint arXiv:2503.05598 (2025).

[Doucet(09)] Doucet Arnaud, Johansen Adam. "A tutorial on particle filtering and smoothing: Fifteen years later." (2009).

[Chen(23)] Chen, Xiongjie, and Yunpeng Li. "An overview of differentiable particle filters for data-adaptive sequential Bayesian inference." arXiv preprint arXiv:2302.09639 (2023).

[Zheng(17)] Zheng, Xun, et al. "State space LSTM models with particle MCMC inference." arXiv preprint arXiv:1711.11179 (2017).

References

[Brossard et al., 2020] Brossard, M., Barrau, A., and Bonnabel, S. (2020). Ai-imu dead-reckoning. *IEEE Transactions on Intelligent Vehicles*, 5(4):585–595.

[Daum and Huang, 2011] Daum, F. and Huang, J. (2011). Particle degeneracy: root cause and solution. In Kadar, I., editor, *Signal Processing, Sensor Fusion, and Target Recognition XX*, volume 8050, page 80500W. SPIE.

[Daum et al., 2010] Daum, F., Huang, J., and Noushin, A. (2010). Exact particle flow for nonlinear filters. In Kadar, I., editor, *Signal Processing, Sensor Fusion, and Target Recognition XIX*. SPIE.

[Ding and Coates, 2012] Ding, T. and Coates, M. J. (2012). Implementation of the daum-huang exact-flow particle filter. In *2012 IEEE Statistical Signal Processing Workshop (SSP)*. IEEE.

[Evangelidis and Parker, 2019] Evangelidis, A. and Parker, D. (2019). *Quantitative Verification of Numerical Stability for Kalman Filters*, page 425–441. Springer International Publishing.

[Hu and van Leeuwen, 2021] Hu, C. and van Leeuwen, P. J. (2021). A particle flow filter for high-dimensional system applications. *Quarterly Journal of the Royal Meteorological Society*, 147(737):2352–2374.

[Johansen, 2009] Johansen, A. (2009). A tutorial on particle filtering and smoothing: Fifteen years later.

[Li and Coates, 2017] Li, Y. and Coates, M. (2017). Particle filtering with invertible particle flow. *IEEE Transactions on Signal Processing*, 65(15):4102–4116.

[Poterjoy, 2015] Poterjoy, J. (2015). A localized particle filter for high-dimensional nonlinear systems. *Monthly Weather Review*, 144(1):59–76.

[Roux et al., 2021] Roux, A., Changey, S., Weber, J., and Lauffenburger, J.-P. (2021). Cnn-based invariant extended kalman filter for projectile trajectory estimation using imu only. In *2021 International Conference on Control, Automation and Diagnosis (ICCAD)*, page 1–6. IEEE.

[Singh et al., 2023] Singh, H., Chattopadhyay, A., and Mishra, K. V. (2023). Inverse extended kalman filter—part i: Fundamentals. *IEEE Transactions on Signal Processing*, 71:2936–2951.

[Zhou et al., 2022] Zhou, H., Zhao, Y., Xiong, X., Lou, Y., and Kamal, S. (2022). Imu dead-reckoning localization with rnn-iekf algorithm. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, page 11382–11387. IEEE.